

Dynamic Video to Interactive Curriculum Engine

Project Proposal

Genesys Research Lab — FAST National University

Aryan Zia · Muhammad Zubair Ilyas · Muhammad Ahmed

July 2026

1. EXECUTIVE SUMMARY

The **Interactive Curriculum Engine** is an AI-powered platform that automatically converts any technical tutorial video into a structured, interactive, and adaptive learning session. Given a YouTube URL or an uploaded video file, the system produces a complete curriculum — segmented lessons, concept maps, checkpoints, and graded exercises — without any manual authoring effort. The platform targets the widespread problem of *tutorial hell*: developers who passively consume hundreds of hours of free video content yet cannot reproduce the work independently. By generating exercises that require transfer of understanding rather than recall, and by adapting difficulty to each learner's performance in real time, the engine transforms passive viewing into active, measurable practice.

2. PROBLEM STATEMENT

Free technical tutorials (YouTube, Loom, recorded lectures) are abundant, yet passive viewing rarely triggers the independent reasoning required for skill acquisition. Learners re-watch the same content, copy code without understanding it, and stall when the instructor's exact example is absent. The alternative — manually authored interactive courses — is expensive to produce and decays rapidly as technologies evolve. No scalable, automated solution currently bridges the gap between raw video content and structured, adaptive learning.

3. PROPOSED SOLUTION

The engine ingests a video, runs a multi-stage AI pipeline, and delivers a fully interactive curriculum through a web application. Key differentiators:

- Automatic curriculum generation — no instructor effort required after upload.
- Code-grounded exercises — coding and debugging tasks are only generated for segments where on-screen code evidence is confirmed by the vision pipeline.
- Transfer-focused questions — exercises use a different context than the instructor's example, preventing copy-paste solutions.
- Real-time adaptive difficulty — IRT 3PL model updates per-concept mastery after every checkpoint attempt.

- Signal video — an auto-generated cinematic summary (narrated slides) is produced alongside the curriculum for rapid review.
- Multi-tenant architecture — row-level data isolation from day one, suitable for institutional deployment.

4. SYSTEM ARCHITECTURE

The system is organised as a monorepo with four applications (API, Worker, Web, Remotion renderer) and three shared libraries (contracts, shared utilities, AI pipeline). Heavy generation runs as a Celery DAG on a Redis broker; session-time calls are limited to fast evaluation only (target < 2 s p95). Real-time progress events are delivered via Server-Sent Events (SSE).

4.1 AI Pipeline Modules

Module	Name	Description
M1	Ingestion	YouTube (yt-dlp) & file upload; format/size validation; WAV extraction; MinIO storage
M2	Transcript	Whisper large-v3 ASR + pyannote diarisation; word-level timestamps
M3	Vision	Frame sampling, PaddleOCR/TrOCR code OCR, scene detection; < 2 min / 10-min video
M4	Segmentation	Hybrid: pause signals + embedding shift + slide events + LLM refinement
M5	Concept Graph	Directed concept graph with prereq/related edges; pgvector embeddings
M6	Checkpoints	Topic-boundary placement; difficulty-tuned spacing; content-type gating
M7	Exercise Gen	MCQ, coding challenge, debug task, conceptual Q via GPT-4o / Llama 3.1 70B
M8	Test Gen	Auto visible + hidden test cases; mutation testing; anti-cheat similarity check
M9	Evaluation	judge0 sandbox + LLM-judge; verdict + score + hints; < 2 s p95 latency
M10	Adaptive	IRT 3PL; per-concept mastery updates; dynamic difficulty recommendation
M11	Progress	Session state, resume, streak, heartbeat watch-time, locked review mode
M12	Frontend	Next.js 14 + TypeScript + Tailwind; Monaco editor; Plyr player; SSE notifications
M13	Backend API	FastAPI async; PostgreSQL 16 + pgvector; Alembic; Redis; Row-Level Security
M14	Sandbox	judge0 (MVP) → Firecracker microVM (prod); per-tenant rate limits
M15	Monitoring	Prometheus + Grafana + Sentry; golden-set benchmarks; cost controls

4.2 Technology Stack

Layer	Technology
ASR	Whisper large-v3 · faster-whisper · pyannote · silero-vad
Vision / OCR	PaddleOCR · TrOCR · OpenCV · PySceneDetect · CLIP · Real-ESRGAN
LLMs	GPT-4o (high-value) · Llama 3.1 70B / Qwen2.5-Coder (bulk/fallback)
Signal Video	Google Gemini + Edge-TTS narration · Remotion rendering
Backend	FastAPI 0.111 · Python 3.11 · Celery · Redis 7
Frontend	Next.js 14 App Router · TypeScript · Tailwind CSS · Monaco Editor
Database	PostgreSQL 16 + pgvector · Alembic · Row-Level Security
Storage	MinIO / S3 (tenant-scoped prefixes)

Layer	Technology
Sandbox	judge0 → Firecracker microVM
Infra	Docker Compose (dev) · Kubernetes (prod) · Terraform · GitHub Actions
Monitoring	Prometheus · Grafana · Sentry

5. KEY FEATURES (MVP SCOPE)

Video Ingestion

YouTube URL via yt-dlp and direct file upload (≤ 2 GiB, .mp4/.mov/.mkv/.webm/.avi/.m4v) with drag-and-drop UI.

Interactive Player

YouTube iframe and HTML5 player with exercise overlays that pause playback at checkpoints.

Exercise Types

Multiple-choice questions, coding challenges, debugging tasks, and conceptual questions — all sandboxed and auto-graded.

Difficulty Selector

Easy / Medium / Hard setting tunes checkpoint spacing (150 s / 90 s / 60 s) and exercise complexity (difficulty 1–5).

Adaptive Progression

IRT 3PL model surfaces weak concepts to the progress dashboard and recommends next checkpoint difficulty.

Signal Video

Gemini-narrated + Remotion-rendered cinematic summary and HTML study guide generated per curriculum.

Progress Dashboard

Session resume, learning streak, per-concept mastery, and recommendations powered by pgvector similarity search.

Multi-Tenancy

PostgreSQL Row-Level Security enforces cross-tenant data isolation; all artifacts stored under tenant-scoped MinIO prefixes.

6. IMPLEMENTATION PLAN

The project follows a 13-week phased delivery plan:

Phase	Weeks	Deliverables
0 — Foundations	1	R&D; spike, monorepo setup, Docker Compose, CI, contract lock
1 — Extraction	2–3	Ingestion (M1), Transcript (M2), Vision (M3)
2 — Structure	4–5	Segmentation (M4), Concept Graph (M5), Checkpoints (M6)
3 — Generation	6–8	Exercise Gen (M7), Test Gen (M8), Evaluation (M9)
4 — Interaction	9–11	Frontend (M12), Adaptive (M10), Progress (M11)
5 — Production	12–13	Polish, benchmarking, monitoring (M15), deployment

7. TEAM & RESPONSIBILITIES

Member	Role	Primary Modules
Aryan Zia	AI / ML Lead	M2, M4, M5, M6, M7, M8, M9, M10
Muhammad Zubair Ilyas	Full-Stack Lead	M1, M11, M12, M13, M14, M15
Muhammad Ahmed	Vision & Ingestion	M1, M3

Integration contracts in libs/contracts/ are the shared seam between the AI and full-stack tracks; any change requires sign-off from both leads.

8. ACCEPTANCE CRITERIA & SUCCESS METRICS

Metric	Target
Checkpoint boundary accuracy	≥ 80 % on 5-video golden test set
Coding challenge validity	≥ 90 % pass automated validation
Evaluation engine agreement	≥ 85 % agreement with human rubric
Evaluation latency (p95)	< 2 seconds
Vision pipeline (10-min video)	< 2 minutes on CPU
User satisfaction	≥ 4 / 5 testers rate experience "useful"

9. CURRENT STATUS

The project is at **v0.1.0**. Phase 0 (foundations) is complete: the monorepo, Docker Compose environment, CI/CD pipelines, database schema (15 tenant-scoped tables with RLS and pgvector), and all integration contracts are locked. The active feature branch feat/performance-upload-difficulty is delivering Phases 1–5 work concurrently, with the ingestion pipeline, transcript extraction, vision module, and core API endpoints already implemented and passing integration tests. Nine Architecture Decision Records (ADRs) document all major technical choices, including the hybrid LLM strategy, code sandbox selection, multi-tenancy approach, and async pipeline design.